

COPILOTTRAINING
TECHNICAL BRIEFING

Which Copilot Where?

A Practical Surface Router

Route eligible work by decisive context, review ownership, and the capability the team needs next.

CopilotTraining · Practitioner Tech Talk

🤔 THE CORE QUESTION

"Which Copilot surface owns this work next?"

Start where the decisive context and controls already live.

Route by context, artifact, reviewer, and handoff.

Developer

Choose the surface that keeps implementation context close.

Product + Docs

Explore, shape, and delegate without beginning in an IDE.

Test + Design

Review evidence and move beside the implementation only when needed.

1 shared map

See local, GitHub, remote, and embedded execution.

4 outcome branches

Route without memorizing product names.

A first win

Onboard each role through useful, reviewed work.

TABLE OF CONTENTS

PART 1

Map the Surfaces

See what each surface does

Compare seven experiences and their best-fit work.

PART 2

Start by Role

Choose a useful first task

Onboard every discipline through reviewed outcomes.

PART 3

Build Adoption Maturity

Advance through evidence

Turn usage signals into reviewed capability growth.

PART 4

Compose the Surfaces

Design visible handoffs

Walk a production incident across changing boundaries.

PART 1

Map the Surfaces

Separate where you steer from where work happens, then route with four questions.



Control

Where do you steer?



Execution

Where do tools run?



Artifact

What comes back?

```
Two independent axes
↳ control location × execution location
```

The Copilot Surface Matrix

Rows show where you steer · Columns show where execution or analysis happens

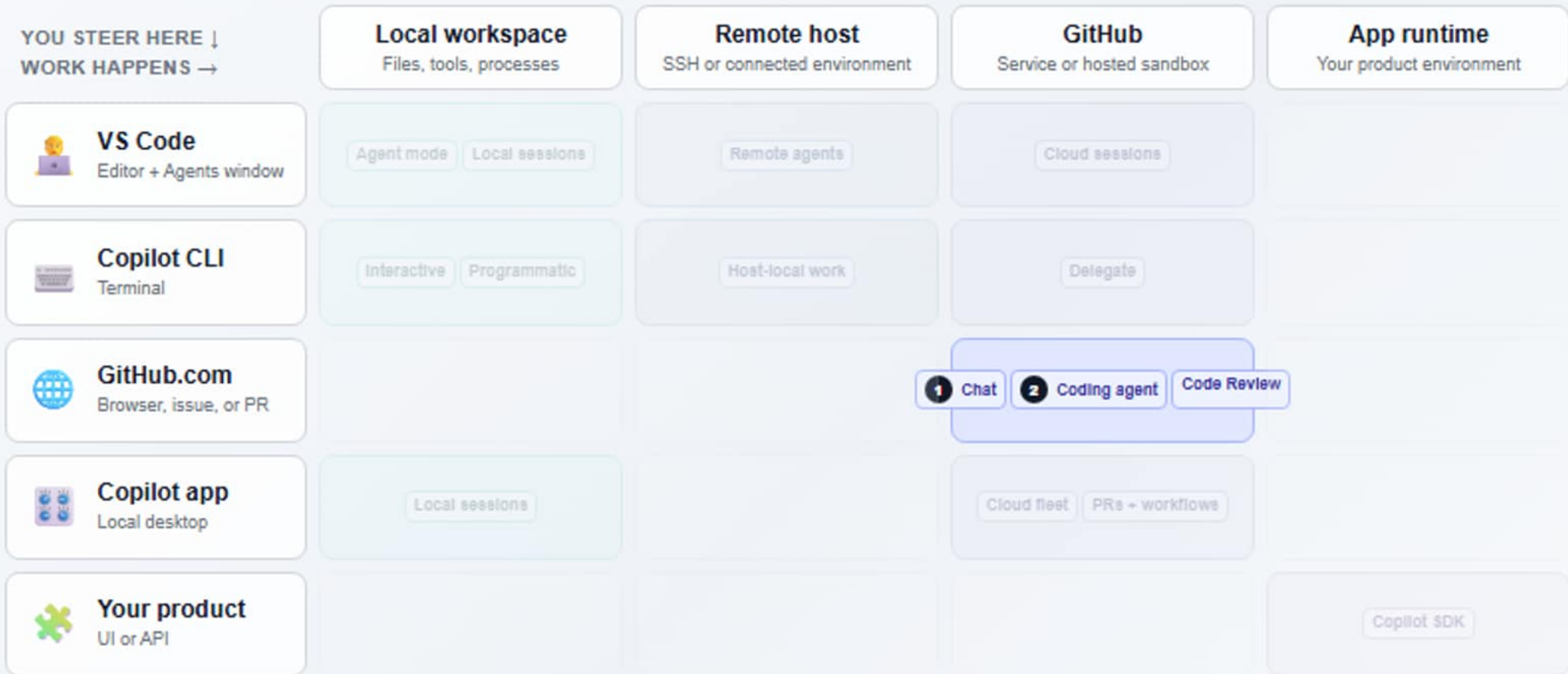
YOU STEER HERE ↓ / WORK HAPPENS →	Local workspace Files, tools, processes	Remote host SSH or connected environment	GitHub Service or hosted sandbox	App runtime Your product environment
VS Code Editor + Agents window	Agent mode · Local sessions	Remote agents	Cloud sessions	
Copilot CLI Terminal	Interactive · Programmatic	Host-local work	Delegate	
GitHub.com Browser, issue, or PR			Chat · Coding agent · Code Review	
Copilot app Local desktop	Local sessions		Cloud fleet · PRs + workflows	
Your product UI or API				Copilot SDK

Off-diagonal cells are the key: your control surface can steer work somewhere else.

SCENARIO · PRODUCT MANAGER

PM: Understand → Shape → Delegate

Stay in GitHub: learn the repository, define acceptance, then hand off a bounded issue



Product manager follows the numbered route.

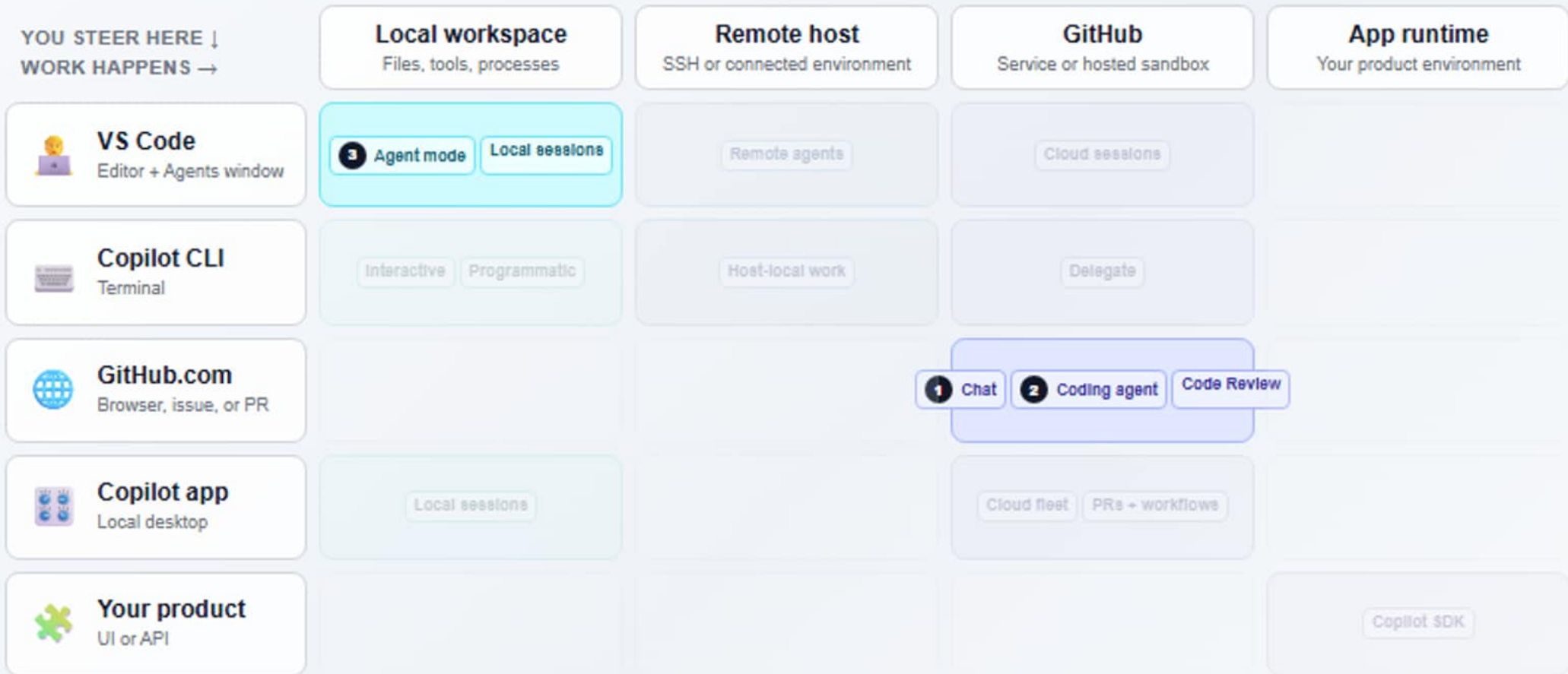
Artifact: Evidence-backed issue + draft pull request

SCENARIO · DOCUMENTATION

3 of 8

Docs: Discover → Draft → Refine

Use repository context before editing; keep editorial acceptance with the docs owner



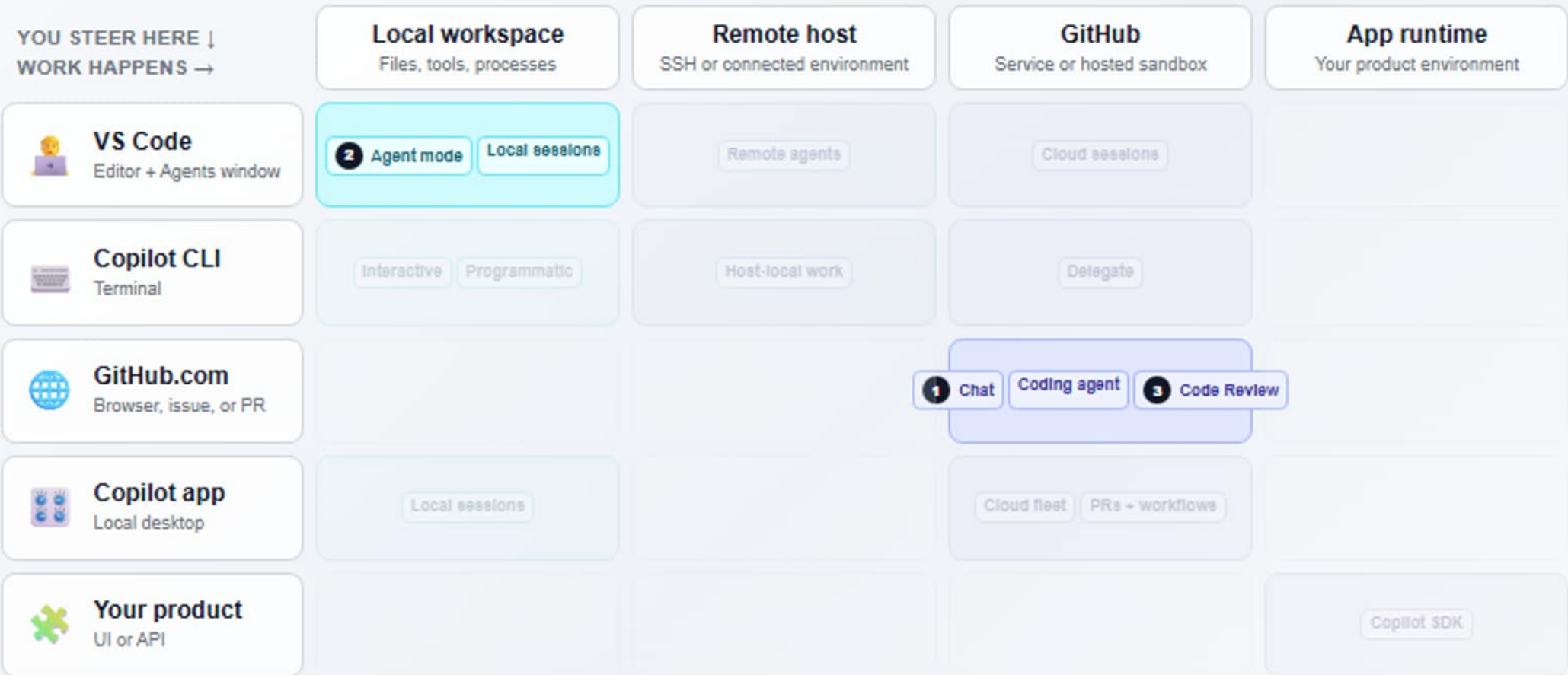
Docs specialist follows the numbered route.

Artifact: Verified documentation pull request

SCENARIO · PRODUCT DESIGN

Designer: Explore → Iterate → Review

Start with implementation facts, then move beside the UI when visual feedback becomes decisive



Product designer follows the numbered route.

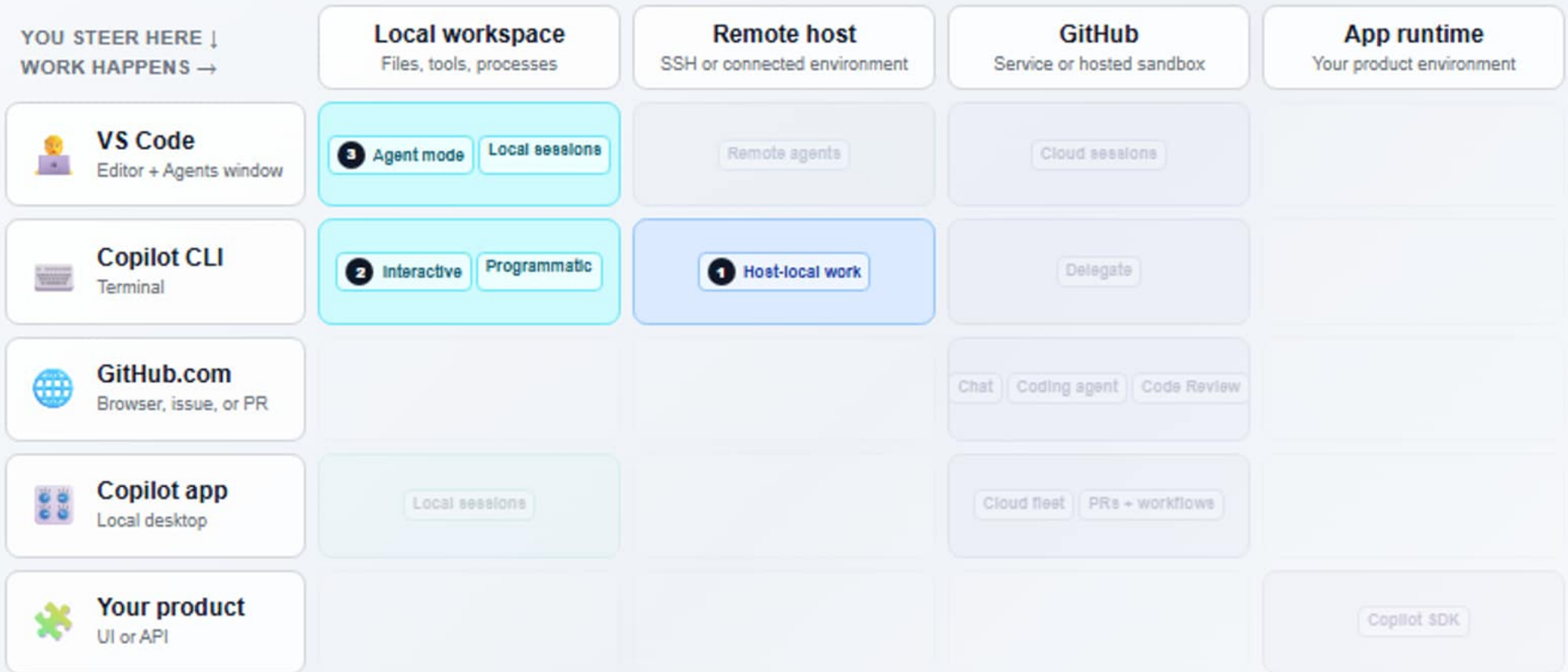
Artifact: Implemented UI change with visual acceptance evidence

SCENARIO · PLATFORM OPERATIONS

5 of 8

Platform Team: Diagnose → Prove → Encode

Start in the CLI when runtime state is decisive; move only after the remediation becomes repository work



Platform engineer · SRE · cloud operations follows the numbered route.

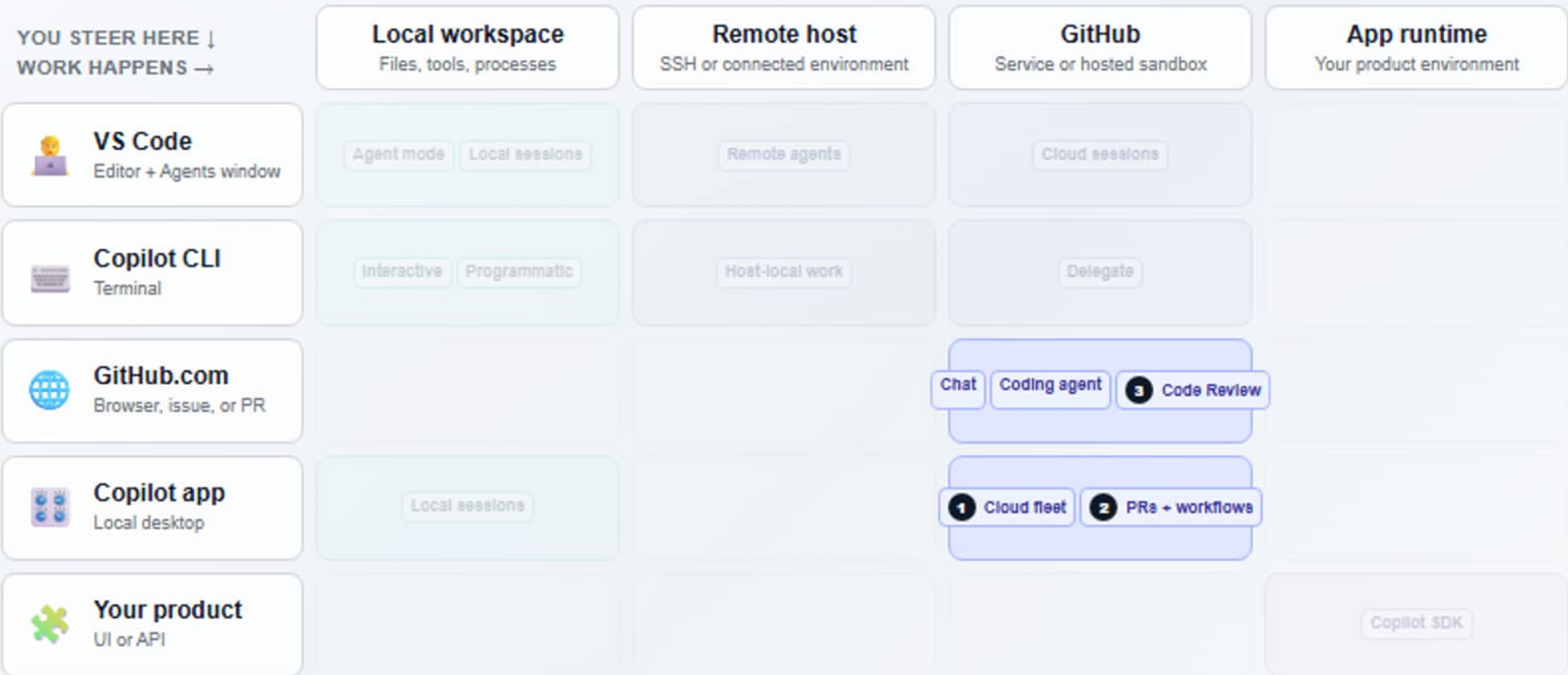
Artifact: Validated diagnosis + reviewable infrastructure change

SCENARIO · DELIVERY LEADERSHIP

6 of 8

Delivery Lead: Dispatch → Monitor → Accept

Start in the Copilot app when several bounded streams are ready and coordination is the work



Engineering manager · **release lead** · **program lead** follows the numbered route.

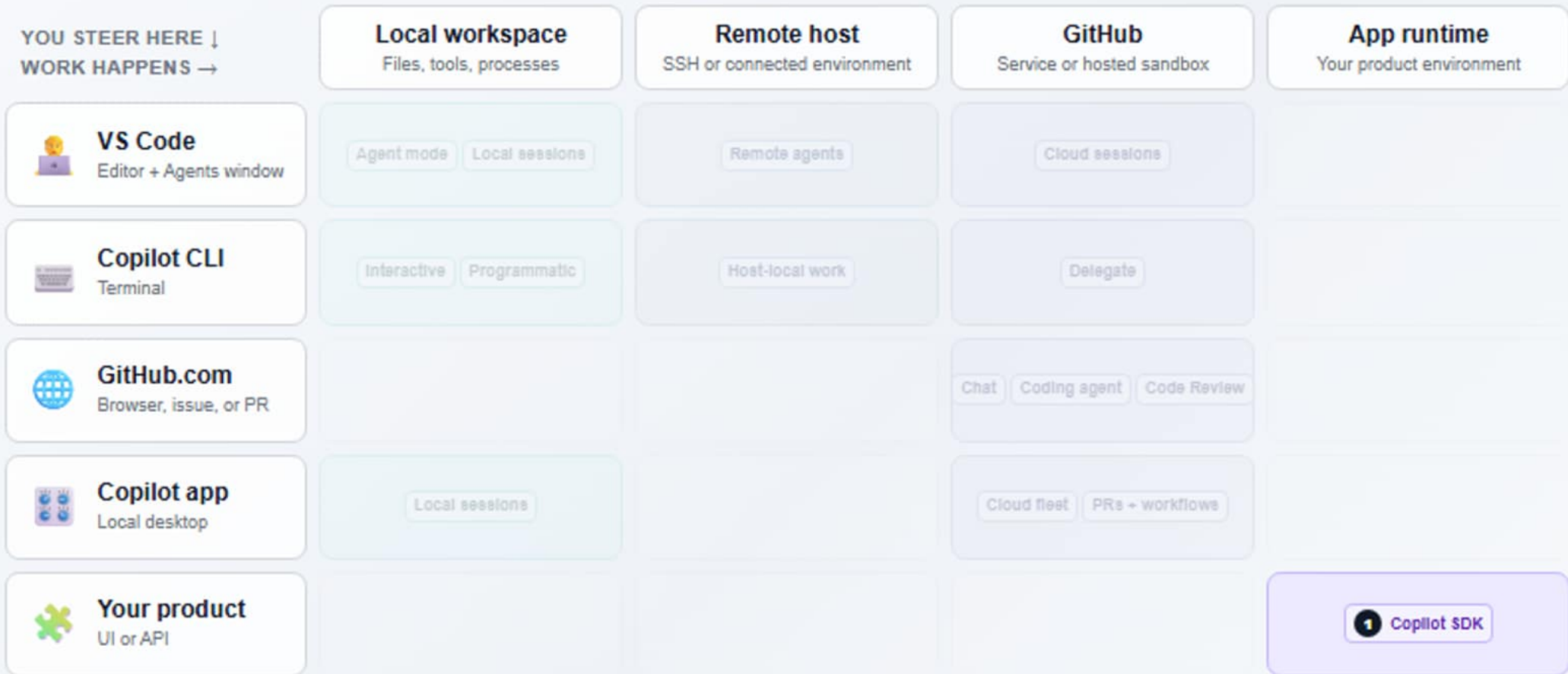
Artifact: Coordinated set of reviewed pull requests

SCENARIO · EMBEDDED PRODUCT

7 of 8

Platform Product Team: Embed the Workflow

Start with the SDK when Copilot must live behind your trigger, tools, policy, and product experience



Platform engineer · **internal tools team** · **support automation owner** follows the numbered route.

Artifact: Governed agent workflow inside the product

Route by Outcome, Then by Context

Your role suggests a first branch; the artifact and decisive context choose the surface

START WITH THE WORK, NOT THE JOB TITLE

What must exist when this work is done?

Understand or shape

Need an explanation, brief, or better issue?

Often starts with: PM · designer · support

Repository and GitHub context
GitHub Chat
Explore without a local checkout

Review or accept

Need findings or evidence before approval?

Often starts with: tester · docs · reviewer

PR or changed code
Code Review
Find risks; humans accept

Create or change

Where is the context you cannot cheaply recreate?

Often starts with: developer · platform · docs · design

Files, UI, diagnostics
VS Code
Steer interactively

Shell, host, runtime
Copilot CLI
Work beside live evidence

Bounded issue
Coding agent
Return a draft PR

Scale or repeat

Are you coordinating work or embedding capability?

Often starts with: platform · lead · product

Many independent streams
Copilot app
Coordinate the fleet

Inside your product
Copilot SDK
Own the runtime contract

When the artifact or decisive context changes, return to the tree and route the next phase.

PART 2

Start by Role

Give every role one useful first task, one reviewable artifact, and one clear handoff.

1

First task

Choose low-risk real work

📦

Artifact

Produce something reviewable

➡️

Handoff

Move when context changes

```
Role-based onboarding
↳ First win → evidence → next surface
```

First Wins for PM and Documentation

Product manager · GitHub Chat

First task
Map an unfamiliar feature from repository evidence

Ask
Explain behavior, dependencies, open issues, and user impact

Keep
A cited brief plus acceptance criteria for one bounded issue

Move when
The issue is ready: coding agent returns a draft PR

Docs specialist · Chat → agent

First task
Find docs made stale by one merged product change

Ask
Identify affected pages, source evidence, and examples

Keep
A verified docs PR reviewed by the documentation owner

Move when
Tone or structure needs judgment: refine in VS Code

First Wins for Test and Design

🧪 Tester · Code Review

First task
Review one bounded PR for missing tests and edge cases

Ask
Focus on changed behavior, failure paths, and test evidence

Keep
Findings plus a human acceptance or rejection decision

Move when
A finding needs reproduction: use VS Code or CLI

🎨 Designer · Chat → VS Code

First task
Trace one interface from design intent to implementation

Ask
Locate components, tokens, states, and accessibility behavior

Keep
An implemented UI change with visual acceptance evidence

Move when
The PR is ready: Code Review checks implementation risk

First Wins for the Copilot App and SDK

Copilot app · fleet owners

Who
Engineering managers, release leads, and program leads

First task
Coordinate three independent, review-ready backlog items

Keep
Visible status, bounded escalation, and a reviewed PR set

Do not start here
When the work still needs discovery or close code steering

Copilot SDK · product owners

Who
Platform engineers, internal tools teams, and automation owners

First task
Embed one proven workflow behind a real product trigger

Keep
Structured output, policy controls, telemetry, and escalation

Do not start here
For a one-off task that an existing surface already handles

PART 3

Build Adoption Maturity

Use telemetry to locate the starting point and reviewed capability evidence to advance.



Baseline

Locate current behavior



Evidence

Review real artifacts



Threshold

Define quality locally

```
Configurable adoption cadence
↳ 4 phases → reviewed evidence
```

Four Phases, One Evidence Gate

Telemetry locates the start; reviewed outcomes earn advancement

0

Activate

Complete one relevant task with human review

1

Standardize

Make interactive help repeatable and repo-aware

2

Delegate

Deliver one bounded asynchronous artifact

TARGET

3

Orchestrate

Coordinate independent work as a governed system

Maturity belongs to a workflow, not permanently to a person.

The Capability Scorecard

📌

Baseline

Record recent behavior and the delivered outcome for this workflow.

📄

Reviewed artifacts

Count normal-work outputs accepted by the named reviewer.

🛡️

Quality guardrail

Define the local threshold and escalation limit before advancing.

🎯

Next-phase target

Name the next capability the team must demonstrate.

PART 4

Compose the Surfaces

Walk one production diagnosis across CLI evidence, a VS Code fix, and delegated follow-up.



Diagnose

Collect runtime evidence



Fix

Change with repo context

Delegate

Continue with review

```
Prepared incident walkthrough
↳ Evidence → fix → follow-up
```

Diagnose Where the Runtime Lives

Keep host-local evidence on the affected machine

```
$ copilot
> "Inspect the failing process and host-local logs. Do not change code yet."
Copilot CLI:
  Read process state and recent logs
  Separate the runtime symptom from the repository defect
  Return evidence and a bounded remediation boundary
```

- ✅ Runtime evidence stays on the affected host
- ✅ A repository configuration defect becomes the next artifact

Control: immediate human steering at the prompt

Restore Now, Prevent the Repeat

The incident ends after the validated fix; prevention is a separate, optional work item

✗ Single-surface attempt

1. **Copy logs into the editor**
   Runtime context is reconstructed
2. **Change files from the shell**
   Visual diff and diagnostics are weak
3. **Keep polishing during recovery**
   Prevention work delays restoration
4. **Skip prevention work**
   The same failure can return

↻ Recovery slows or prevention disappears

✓ Composed route

1. **Diagnose in CLI**
   Incident owner reviews host evidence
2. **Fix and validate in VS Code**
   Developer reviews the workspace diff
3. **Close the incident**
   The urgent route ends when service is restored
4. **Optionally delegate prevention**
   Coding agent adds a regression test and runbook update in a separate draft PR

✓ Service restored + optional prevention PR

Coding agent does not redo the fix; it handles bounded follow-up work asynchronously

Three Reusable Handoff Patterns



Explore → Delegate → Review

A repository question becomes a bounded issue and reviewed PR.

- Chat
- Coding agent
- VS Code



Diagnose → Encode → Productize

An interactive operational proof becomes reusable capability.

- CLI
- VS Code
- SDK



Plan → Fan Out → Integrate

Independent tasks move through coordinated sessions to review.

- VS Code or CLI
- Copilot app
- PR review

Boundary Signals Tell You When to Move

Change surfaces only when the work contract changes

Local state	Keep uncommitted files, services, and credentials local	`VS Code/CLI`
Unbounded	Explore and plan before asynchronous delegation	`interactive`
Repeatable	Evaluate an embedded service for proven event-driven work	`SDK`
Many streams	Coordinate independent sessions from one control plane	`Copilot app`
Understand	Use repository context without introducing mutation	`GitHub Chat`

At every transition: name the artifact, evidence, reviewer, and escalation trigger.

From Tool Choice to Explicit Work Routing

Before

- Choose by habit or product familiarity
- Move context manually between tools
- Leave acceptance ownership implicit
- Treat adoption as usage volume

After

- Choose by execution and decisive context
- Move work when a boundary changes
- Name the reviewer at every handoff
- Advance through reviewed capability evidence

7	4	3
surface experiences	routing questions	lifecycle patterns

✅ WHAT YOU CAN DO TODAY

Today

- Name the decisive context for one task
- Identify its delivery artifact
- Name the accepting reviewer

This Week

- Test the four routing questions
- Document one visible handoff trigger
- Review one capability artifact

This Month

- Baseline your team across four phases
- Set local quality thresholds
- Practice all three lifecycle patterns

🔑 Key Takeaway

Start the work where its strongest context and controls already live.

REFERENCES

Official Documentation

GitHub Copilot documentation
Product-wide concepts and workflows.

GitHub Copilot in VS Code
Editor integrations, chat, and agent workflows.

About GitHub Copilot CLI
Terminal-native capabilities and operating model.

GitHub Copilot Code Review
AI findings for pull requests and changed code.

Agents and Embedding

GitHub Copilot app
Desktop session management and coordination.

GitHub Copilot SDK
Agent capabilities for applications and services.



Which Copilot Where?

ROUTE BY CONTEXT. HAND OFF BY BOUNDARY. REVIEW BY CONTRACT.

7 experiences
One visible landscape

4 questions
One practical router

3 patterns
One composable lifecycle

Which task will your team route differently next?